

React com Vite: JSX e primeiros componentes

Aula prática · do HTML em string ao componente reutilizável

JSX

CodeSandbox

Componentes



O roteiro da aula

1 Do HTML em string ao JSX

A ponte entre o que você já sabe e o React

2 O que é o Vite (e por que não Webpack/CRA)

Diagrama navegador → Vite → React → React DOM

3 Regras do JSX + 4 erros para prever

Você adivinha, depois a gente roda

4 Árvore de componentes, export/import

Programação ao vivo nº 2

5 Prática: Painel de Interesses

Cabeçalho, Destaque, cartões e Rodapé

6 Fechamento e gancho para props

Checklist do que você já sabe fazer

Função que devolve uma string de HTML

antigo.js

```
function cartao(titulo, texto) {  
  return (  
    '<div class="cartao">' +  
    '<h3>' + titulo + '</h3>' +  
    '<p>' + texto + '</p>' +  
    '</div>'  
  );  
}  
  
document.querySelector('#app').innerHTML =  
  cartao('Games', 'Level design e narrativa');
```

Dor 1

O editor enxerga **texto**, não HTML: sem autocompletar, sem aviso de tag aberta.

Dor 2

Concatenar aspas e variáveis vira sopa de + e '
'.

Dor 3

Atualizar a tela é apagar tudo e escrever de novo com `innerHTML`.

Função → Componente

Função comum (string)

antigo.js

```
function cartao(titulo) {  
  return '<h3>' + titulo + '</h3>';  
}
```

mesma ideia

Componente React (JSX)

Cartao.jsx

```
function Cartao() {  
  return <h3>Games</h3>;  
}
```

Nome com letra MAIÚSCULA

Cartao, Cabecalho, Destaque — é assim que o React distingue componente de tag html.

Devolve marcação, não texto

O que está depois do return é JSX: o editor entende e avisa erros.

Vira uma tag sua

Depois você escreve <Cartao /> em qualquer lugar da aplicação.

O que é o Vite?

Vite = servidor de desenvolvimento + tradutor de código. Ele fica entre os seus arquivos `.jsx` e o navegador.

1 Serve os arquivos

Sobe um endereço local (localhost) que entrega o seu projeto ao navegador.

2 Traduz o JSX

O navegador não entende `<h3>` dentro do JavaScript. O Vite converte para chamadas comuns de JS.

3 Atualiza na hora

Salvou o arquivo, a tela muda em milissegundos — sem recarregar a página inteira.

Analogia

Restaurante à la carte: o Vite prepara **só o prato pedido**, no momento em que a tela precisa dele.

Por baixo do capô

O Vite usa **módulos ES nativos** do navegador durante o desenvolvimento e o tradutor **esbuild**, escrito em Go, que é dezenas de vezes mais rápido que tradutores em JavaScript.

No **CodeSandbox** o Vite já vem rodando dentro do navegador: você não instala nada, só escolhe o modelo React + Vite.

Por que Vite e não Webpack/CRA?

Webpack / Create React App

- Empacota **todo** o projeto antes de abrir a primeira tela
- Início do servidor: dezenas de segundos a minutos
- Cada "salvar" reprocessa um pacote grande
- Muita configuração para tarefas simples
- CRA foi **descontinuado** pela equipe do React

Vite

- Serve os arquivos **sob demanda**, um a um
- Início do servidor: quase instantâneo
- Ao salvar, troca **só o módulo alterado** na tela
- Configuração mínima: um arquivo pequeno
- Caminho recomendado hoje para começar com React

Em uma frase: **Webpack/CRA cozinham o cardápio inteiro antes de abrir; o Vite abre na hora e cozinha o prato pedido.**

Quem faz o quê



O caminho da informação é sempre o mesmo: **você escreve componente** → **Vite traduz** → **React monta a descrição** → **React DOM desenha** → **navegador exibe**.

index.html → main.jsx → App

1

index.html

A casca da página. Tem uma div vazia com id="root" e um script apontando para /src/main.jsx.

2

main.jsx

Ponto de entrada. Encontra a div #root e pede ao React DOM para desenhar o <App /> ali dentro.

3

App.jsx

Componente raiz. A partir dele nascem Cabecalho, Destaque, Cartao e Rodape.

Tela em branco? Verifique nesta ordem: a div #root existe? o main.jsx foi chamado? o App foi importado com o nome certo?

O ponto de entrada, linha a linha

src/main.jsx

```
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './index.css'

const elemento = document.getElementById('root')

createRoot(elemento).render(<App />)
```

index.html (trecho)

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

createRoot

Cria o ponto de entrada do React dentro da div #root.

./App.jsx

O ponto-barra indica arquivo do seu projeto.

react-dom/client

Sem ponto-barra: pacote instalado.

render(<App />)

Aqui o seu componente vira tela de verdade.

Três regras que você vai usar hoje

1 · Um único elemento

O return devolve UM elemento pai.
Vários irmãos soltos = erro.

```
return (  
  <>  
    <h1>Painel</h1>  
    <p>Interesses</p>  
  </>  
)
```

2 · Toda tag fecha

Tags sem conteúdo fecham sozinhas com barra: img, br, hr, input.

```
  
<br />  
<input type="text" />
```

3 · Chaves = JavaScript

Dentro de { } vale qualquer expressão JavaScript.

```
const nome = 'Ana'  
  
return <h1>Olá, {nome}!</h1>
```

Atributos: o que muda em relação ao HTML

HTML	JSX
class	className
for	htmlFor
onclick	onClick
tabindex	tabIndex

Exemplo correto

```
function Destaque() {  
  const tema = 'Games'  
  
  return (  
    <section className="destaque">  
      {/* comentário dentro do JSX */}  
      <h2>Em alta: {tema}</h2>  
      <label htmlFor="busca">Buscar</label>  
      <input id="busca" type="text" />  
    </section>  
  )  
}
```

Regra de bolso: se o atributo tem duas palavras no HTML, no JSX a segunda começa com maiúscula.

O que este código vai reclamar?

Cabecalho.jsx

```
function Cabecalho() {  
  return (  
    <h1>Painel de Interesses</h1>  
    <p>Turma de Front-end</p>  
  )  
}
```

🤔 Escreva no chat o que você acha que vai acontecer.



Clique para ver a resposta

Peça os palpites da turma antes de revelar

Dois problemas escondidos aqui

● ● ● Cartao.jsx

```
function Cartao() {  
  return (  
    <div class="cartao">  
        
      <h3>Games</h3>  
    </div>  
  )  
}
```



Clique para ver a resposta

Peça os palpites da turma antes de revelar

Ao vivo: primeiro componente

- 1 Criar o sandbox pelo modelo React + Vite
- 2 Passear pelos arquivos: index.html, main.jsx, App.jsx
- 3 Editar o App e ver a tela mudar na hora
- 4 Criar o componente Cabecalho em um arquivo novo
- 5 Importar e usar `<Cabecalho />` dentro do App

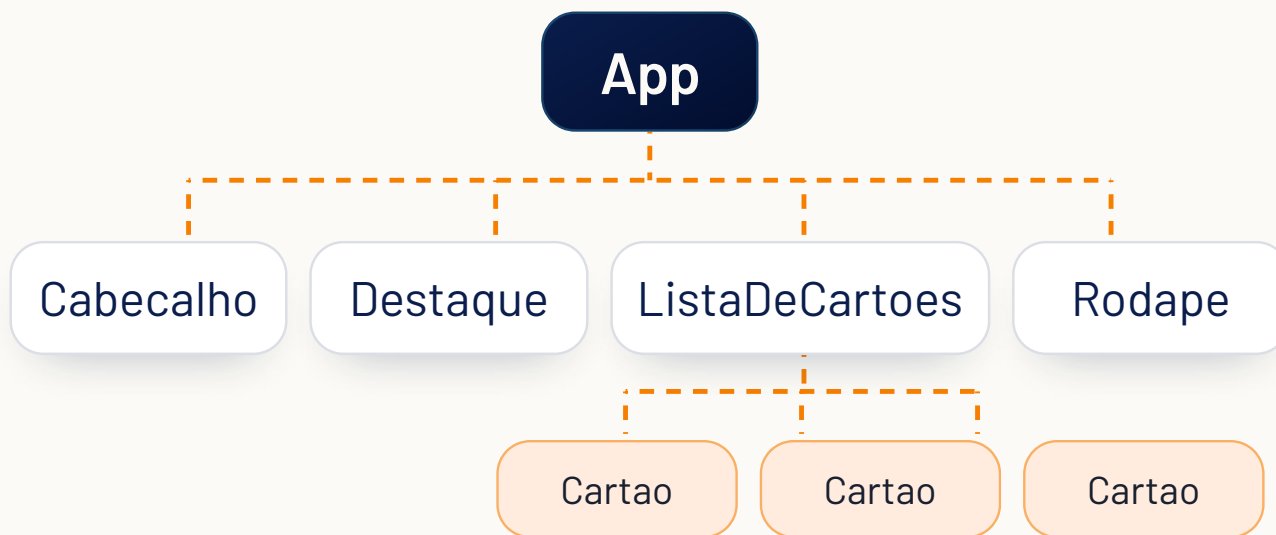
src/Cabecalho.jsx

```
export default function Cabecalho() {  
  return (  
    <header className="cabecalho">  
      <h1>Painel de Interesses</h1>  
      <p>Turma de Front-end Frameworks</p>  
    </header>  
  )  
}
```

src/App.jsx

```
import Cabecalho from './Cabecalho.jsx'  
  
export default function App() {  
  return (  
    <div className="app">  
      <Cabecalho />  
    </div>  
  )  
}
```

A aplicação é uma árvore



5 componentes diferentes na tela — Cartao aparece três vezes, mas o arquivo é um só.

Guarde isso

Três cartões idênticos com conteúdos diferentes é exatamente o problema que **props** resolve — assunto do fim da aula.

Como um componente sai de um arquivo e entra em outro

Exportação padrão (default)

● ● ● Rodape.jsx → App.jsx

```
// Rodape.jsx
export default function Rodape() {
  return <footer>© 2026 · UNINASSAU</footer>
}

// App.jsx
import Rodape from './Rodape.jsx' // sem chaves
```

Exportação nomeada

● ● ● ui.jsx → App.jsx

```
// ui.jsx
export function Badge() { return <span>Novo</span> }
export function Linha() { return <hr /> }

// App.jsx
import { Badge, Linha } from './ui.jsx' // com chaves
```

Um por arquivo

É a nossa convenção hoje: um componente, um arquivo, mesmo nome.

Nome com maiúscula

Arquivo Cartao.jsx, componente Cartao, uso <Cartao />.

Caminho relativo

./ para o mesmo diretório, ../ para subir um nível.

A tela fica em branco. Por quê?

● ● ● Destaque.jsx / App.jsx

```
// Destaque.jsx
export default function Destaque() {
  return <section>Em alta: Games</section>
}
```

```
// App.jsx
import { Destaque } from './Destaque.jsx'
```



Clique para ver a resposta

Peça os palpites da turma antes de revelar

Nada aparece e o texto sai errado

App.jsx

```
const titulo = 'Games'

function cartao() {
  return <h3>titulo</h3>
}

export default function App() {
  return <cartao />
}
```



Clique para ver a resposta

Peça os palpites da turma antes de revelar

Cinco sinais de que é hora de extrair



Repetição

O mesmo bloco apareceu 2 ou 3 vezes.



Tem nome

Você consegue batizar: Cartao, Rodape, Destaque.



Tamanho

O arquivo passou de uma tela de rolagem.



Fronteira visual

É um pedaço com começo e fim claros na página.



Reuso previsto

Vai aparecer em outra página depois.

Contraexemplo: **não** crie um componente só para envolver um parágrafo único. Extrair sem motivo espalha o código e dificulta a leitura.

Ao vivo: extraindo Destaque e Cartao

● ● ● antes · App.jsx inchado

```
export default function App() {
  return (
    <div>
      <header>...</header>
      <section>...</section>
      <div className="cartao">...</div>
      <div className="cartao">...</div>
      <div className="cartao">...</div>
      <footer>...</footer>
    </div>
  )
}
```



● ● ● depois · App.jsx enxuto

```
import Cabecalho from './Cabecalho.jsx'
import Destaque from './Destaque.jsx'
import Cartao from './Cartao.jsx'
import Rodape from './Rodape.jsx'

export default function App() {
  return (
    <div className="app">
      <Cabecalho />
      <Destaque />
      <Cartao />
      <Cartao />
      <Cartao />
      <Rodape />
    </div>
  )
}
```

Painel de Interesses

Cabecalho Título do painel e uma linha de subtítulo.

Destaque Um bloco maior com o interesse da semana.

Cartao **x3** Mesmo componente usado três vezes seguidas.

Rodape Seu nome, a turma e o ano.

Regras da entrega

Um componente por arquivo · `export default` · nomes com letra maiúscula · nada de estado ou interatividade hoje.

Cabecalho

Destaque

Cartao

Cartao

Cartao

Rodape

Roteiro: primeiro crie os quatro arquivos · depois monte tudo no App e estilize · por fim compare em dupla e corrija os erros.

O que você já sabe fazer

- ✓ Explicar em uma frase o que o Vite faz
- ✓ Percorrer index.html → main.jsx → App
- ✓ Escrever JSX sem os quatro erros de hoje
- ✓ Criar, exportar e importar componentes
- ✓ Decidir quando vale extrair um componente

o problema que fica em aberto

```
<Cartao />
<Cartao />
<Cartao />

// três cartões idênticos...
// e se cada um precisar de um título?
```

Próxima aula

props: passar informação de fora para dentro

```
<Cartao titulo="Games" />
```